University of Glasgow

WORLD CHANGING GLASGOW

Lecture 4
Digital-to-Analogue Converters (DACs) and Direct Memory Access (DMA)

UESTC 2004: Embedded Processors

Dr Mark D. Butala

Some slides were prepared by Dr Yao Sun, Dr Lina Mohjazi, and Dr Ahmad Taha

University of Glasgow 2013 Glasgow College, UESTC

Not everything covered in lecture “will be on the exam”

Refer to the chili pepper scale:

- 🌶️ : Supplemental (unlikely to appear on the exam)
- 🌶️🌶️: Just for fun (not on the exam)
- Otherwise: …

Lesson Plan

Aim

- Use the DAC channels on the NUCLEO L432KC to generate an analogue output, e.g,. PWM
- Conceptual introduction to Direct Memory Access (DMA)

Learning Objectives

By the end of this lesson, you will be able to:

- Write an IRQ handler within the mbed framework
- Recall the characteristics and working principle of a DAC and DMA
- Write C codes to generate an analogue output

Assumed Previous Knowledge

- Basic MCU interfacing techniques
- C programming

Click for Course Specifications & Learning Outcomes

Part 1:

Using an interrupt to handle an asynchronous event

Polling vs. Interruption

- **Polling (轮询)**: the CPU actively interacts with an external device to take an action

- **Interruption (中断)**: an external device requests the CPU to take an action

- Polling is generally simpler to implement and aligns well with *synchronous* events (同步事件, e.g., like sampling)

User Interaction

- Interruption aligns well with *asynchronous* (龙卷风) events

- Example: User presses a button, and the CPU acts

- How are interrupts implemented?

1. A *callback routine* (回调函数) is first associated with an interrupt ID
2. An interrupt request signal asynchronously triggers the callback routine
3. Program execution resumes

Nucleo L432KC Board – InterruptIn Class Reference

Function	Description
InterruptIn **name**(PinName pin)	Create an InterruptIn object, connected to the specified pin.
int **read**() operator **float**()	Read the input, either 0 (logical false) or 1 (logical true).
void **rise**(Callback< void()> func)	Attach a function to call when a rising edge (low to high) is detected.
void **fall**(Callback< void()> func)	Similar to rise, but call the function when the falling edge (high to low) is detected.
void **mode**(PinMode pull)	Set the input pin mode (PullUp, PullDown, PullNone, or PullDefault).
void **enable_irq**() void **disable_irq**()	Turn the interrupt handler on (enable) or off (disable)

LOW to HIGH (**Rising edge**)

HIGH to LOW (**Falling edge**)

Button Press IRQ Example

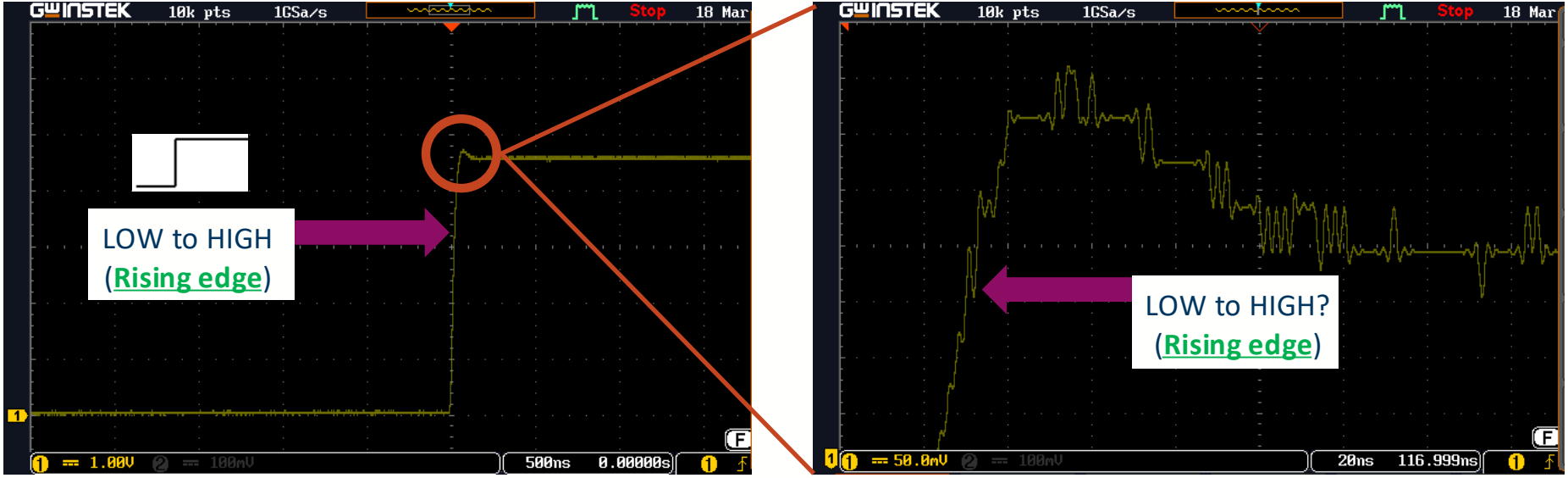
```
1   #include "mbed.h"
2
3   #define PRINTF_RATE      250ms
4
5   DigitalOut led(LED1);
6   InterruptIn button(D6);
7
8   unsigned N_led_toggle;
9
10  void led_toggle() {
11      N_led_toggle++;
12      led = !led;
13  }
```

```
15  int main() {
16      button.mode(PullUp);
17      button.fall(led_toggle);
18      N_led_toggle = 0;
19
20      while (true) {
21          printf("N_led_toggle = %d\n", N_led_toggle);
22          ThisThread::sleep_for(PRINTF_RATE);
23      }
24  }
```

&led_toggle is a pointer to a function with no input arguments (void) or return value (void)

Why Does the Button Trigger Multiple IRQs?

- Measured traces of pullup resistor voltage (D6)



Modified Logic to Software "Debounce" a Pushbutton

Original IRQ handler logic:

1. Toggle the LED state

Clever IRQ handler logic:

1. Check timer: has threshold time elapsed since last IRQ trigger?
2. If yes, toggle the LED state
3. Reset the timer to 0

Button Press IRQ with Debounce Timer Example

```cpp
#include "mbed.h"

#define PRINTF_RATE     250ms
#define TIME_OUT        2000ms
//#define TIME_OUT        100ms

DigitalOut led(LED1);
InterruptIn button(D6);
Timer time_out;

unsigned N_led_toggle;

void led_toggle() {
    if (time_out.elapsed_time() > 0ms && time_out.elapsed_time() < TIME_OUT) {
        led = 0;
        return;
    }
    time_out.reset();
    time_out.start();
    led = 0;
    N_led_toggle++;
}
```

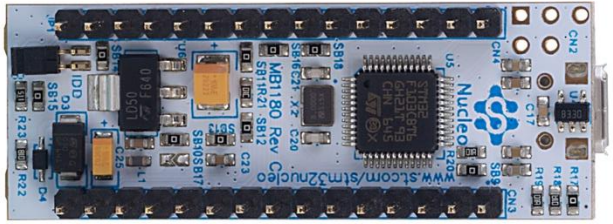
```cpp
int main() {
    button.mode(PullUp);
    button.fall(led_toggle);
    N_led_toggle = 0;
    time_out.reset();
    led = 1;

    while (true) {
        if (time_out.elapsed_time() == 0ms || time_out.elapsed_time() >= TIME_OUT) {
            led = 1;
        }
        printf("N_led_toggle = %d (debounce)\n", N_led_toggle);
        ThisThread::sleep_for(PRINTF_RATE);
    }
}
```

Part 2:

MCU output: PwmOut and AnalogOut

Interfacing with External Devices

Digital processing on MCU



External devices:

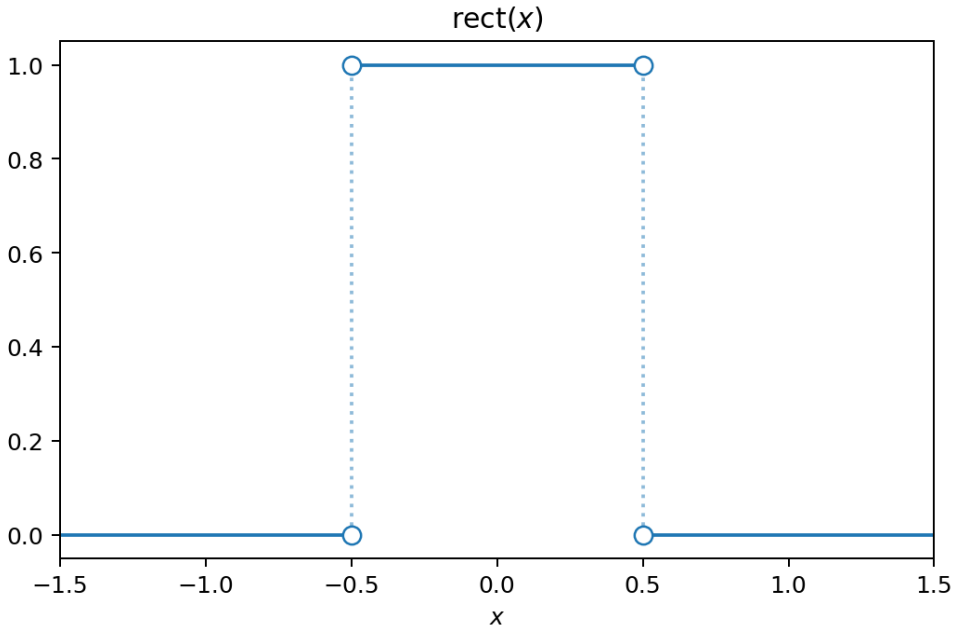
- LED/LCD
- Audio
- Actuator
- Stepper motor
- …

We will consider two types of output signals:

- Pulse width modulation (PWM): value encoded as duty cycle
- Digital to analog converter (DAC): value encoded as amplitude

The Rectangular Function

$$\mathrm{rect}(x) = \begin{cases} 1, & |x| < \frac{1}{2} \\ 0, & |x| > \frac{1}{2} \end{cases}$$

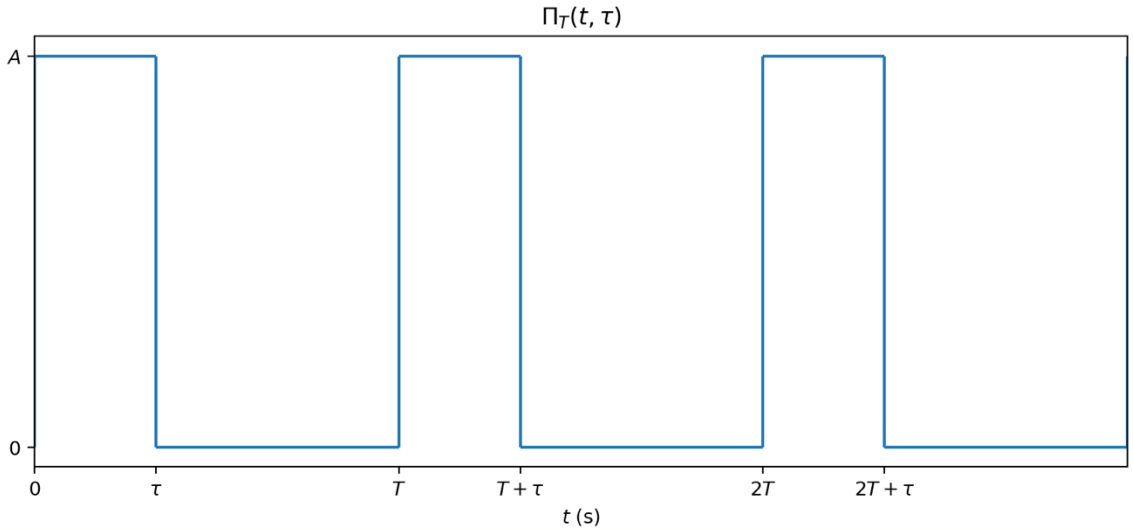


For engineering, the value at $|x| = \frac{1}{2}$ is not important (0, 1, $\frac{1}{2}$, or undefined)

The Rectangular Pulse Train Function

Rectangular pulse train (矩形脉冲序列):

- Repetition of rectangular functions
- Width of rect = τ
- Distance between pulses = T
- Duty cycle (占空比) = $\frac{\tau}{T}$



$$\Pi_T(t,\tau) = A\sum_{i=0}^{N} \mathrm{rect}\left[\left(\frac{t-iT}{\tau}\right) - \frac{1}{2}\right]$$

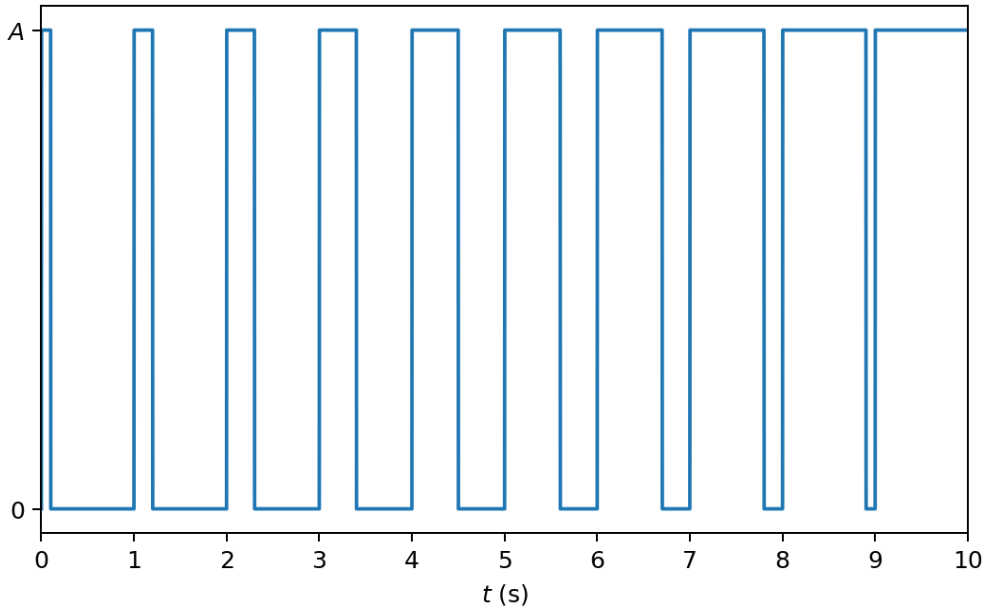
( )

Pulse Width Modulation (脉宽调制, PWM)

$$\mathrm{PWM}(t) = A \sum_{i=0}^{N} \mathrm{rect}\left[\left(\frac{t - iT}{\tau_i}\right) - \frac{1}{2}\right]$$

(🌶)

($\Pi_T(t, \tau)$ with time-dependent pulse width)



$$\tau_i = \frac{i+1}{10}, 0 \le i \le 9$$

$$\text{Duty cycle}(i) = 10(i+1)\%$$

PWM Implementation

Function	Description
PWMOut **name**(PinName pin)	Create a PWMOut object, connected to the specified pin.
void **write**(float value) PWMOut& **operator=**(float value)	Set the output duty-cycle, specified as a percentage (float).
float **read**(void)	Return the current output duty-cycle setting, measured as a percentage.
void **period**(float seconds) void **period_ms**(int ms) void **period_us**(int us)	Set the PWM period, specified in seconds, milliseconds, or microseconds.
void **pulsewidth**(float seconds) void **pulsewidth_ms**(int ms) void **pulsewidth_us**(int us)	Set the PWM pulse width, specified in seconds, milliseconds, or microseconds
void **suspend**() void **resume**()	Turn PWM signal generation off (suspend) or on (resume).

PWM Example: Dimming (Lower Brightness of) an LED

```
#include "mbed.h"

// Blinking rate in milliseconds
#define BLINKING_RATE     500ms

int main() {
    DigitalOut led(LED1);

    PwmOut green_led(D10);
    green_led.period_ms(1);

    unsigned i = 0;
    float step = 0.2;

    while (true) {
        if (i > 5) {
            i = 0;
        }

        green_led = i * step;
        i++;

        led = !led;
        ThisThread::sleep_for(BLINKING_RATE);
    }
}
```

- Alternative: dim by lowering the LED voltage
- What are the advantages of using PWM for dimming?
- How to select the PWM signal period?

Digital to Analog Converter

$$x[n] \longrightarrow \boxed{\text{D/A}} \longrightarrow x(t)$$

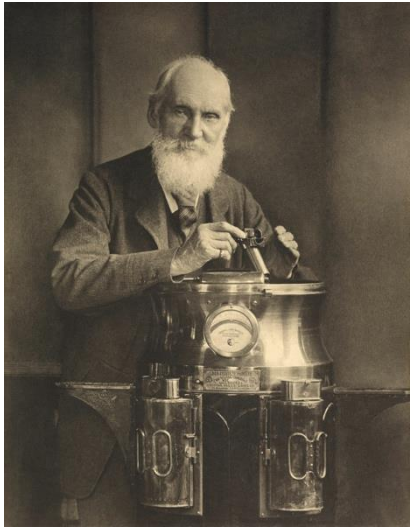
$n \in \mathbb{Z}$, $x[n] \in \mathbb{R}$ $\qquad$ $t \in \mathbb{R}$, $x(t) \in \mathbb{R}$

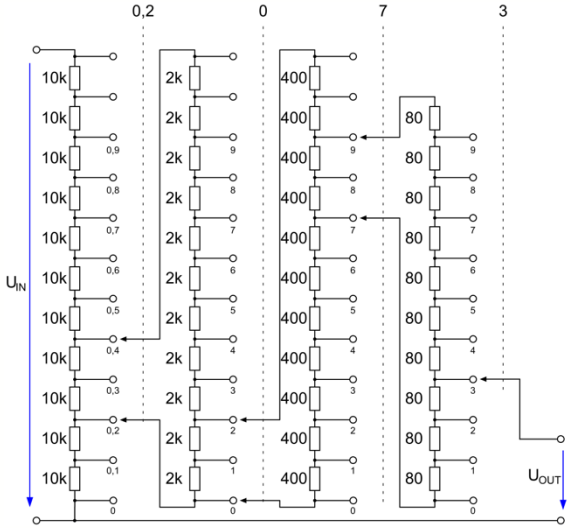
A digital to analog converter (DAC or D/A) has two stages:

1. Convert sampled values to voltages
2. Interpolation (插值)

Values to Voltages: Lord Kelvin's Method (🌶️🌶️)

- High precision DACs use a *Kelvin-Varley divider*
- Expensive (roughly 11 matched resistors 高精度匹配电阻 per stage/digit)



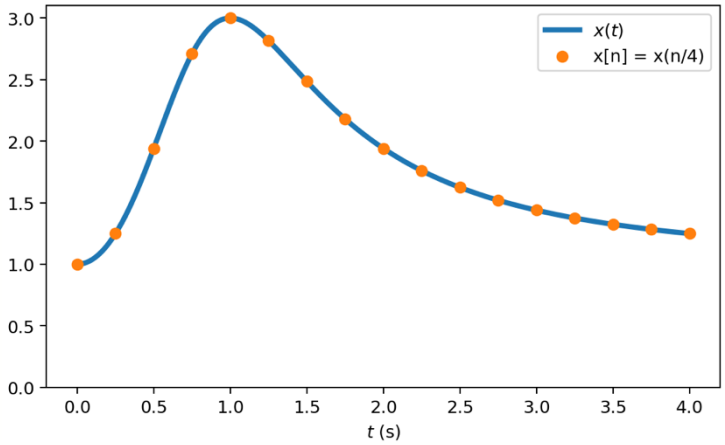


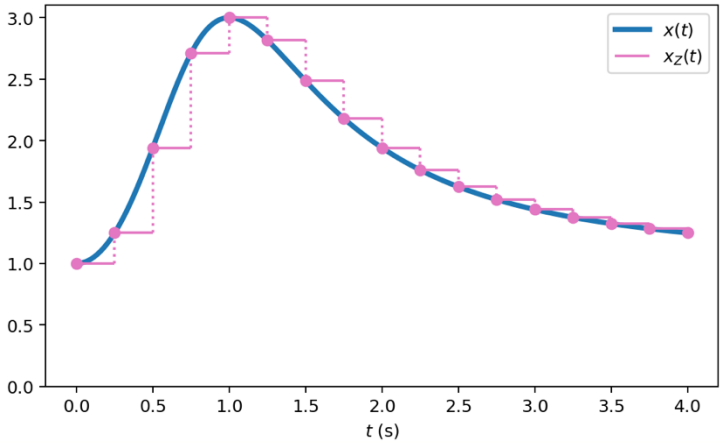
Values to Voltages: Zero-Order Hold (🌶️🌶️

- A zero-order hold converts sampled values to a rect train

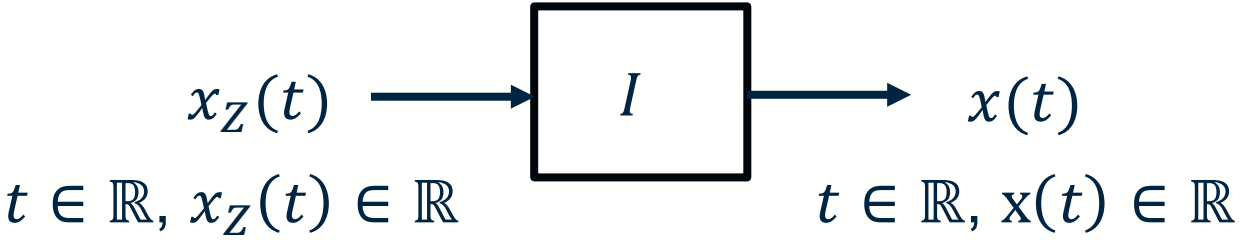
$$x[n] \longrightarrow \boxed{\text{ZOH}} \longrightarrow x_Z(t)$$

$$n \in \mathbb{Z},\ x[n] \in \mathbb{R} \qquad\qquad t \in \mathbb{R},\ x_Z(t) \in \mathbb{R}$$





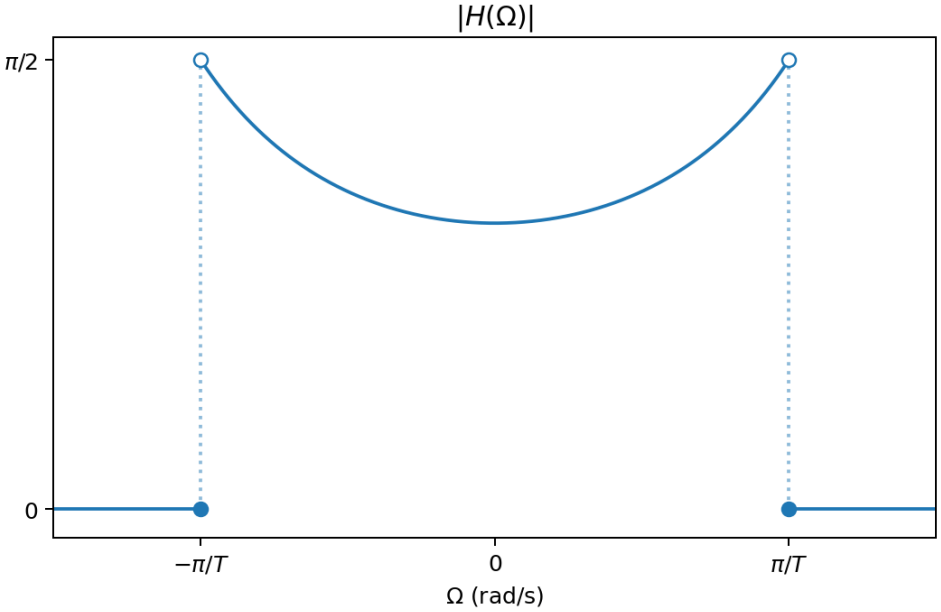
Interpolation (🌶️🌶️)



- The interpolator (I) "smooths" sharp transitions in the zero-order hold (零阶保持器) signal ($x[n]$)
- A "smooth" signal (**平滑信号**) has low frequency, but not high frequency, content
- I is an ANALOG filter (模拟滤波器)!

Interpolation for Zero-Order Hold (🌶️🌶️🌶️)

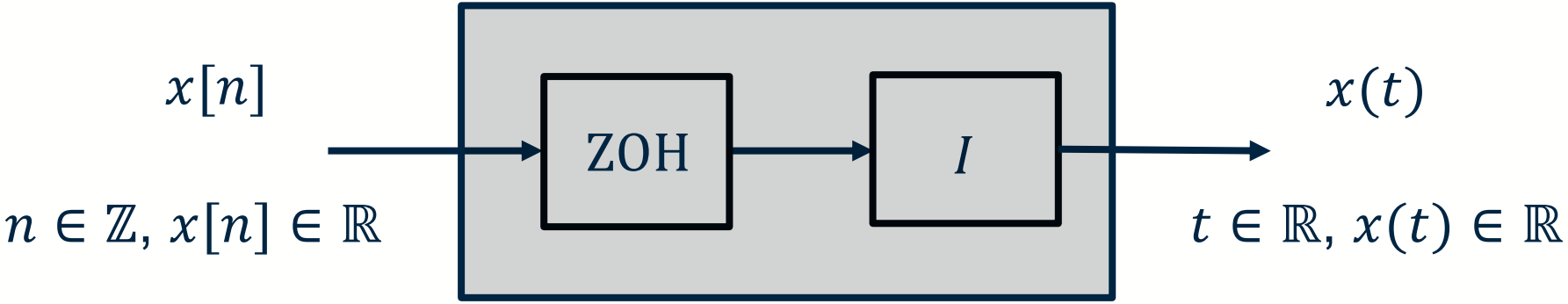
- It can be shown that I is an analog **filter (滤波器)** with frequency response (频率响应):



$$H(\Omega) = \begin{cases} \dfrac{\Omega T/2}{\sin(\Omega T/2)} e^{j\Omega T/2}, & |\Omega| < \pi/T \\ 0, & |\Omega| \geq \pi/T \end{cases}$$

D/A Summary

Digital to Analog Converter (D/A, DAC)



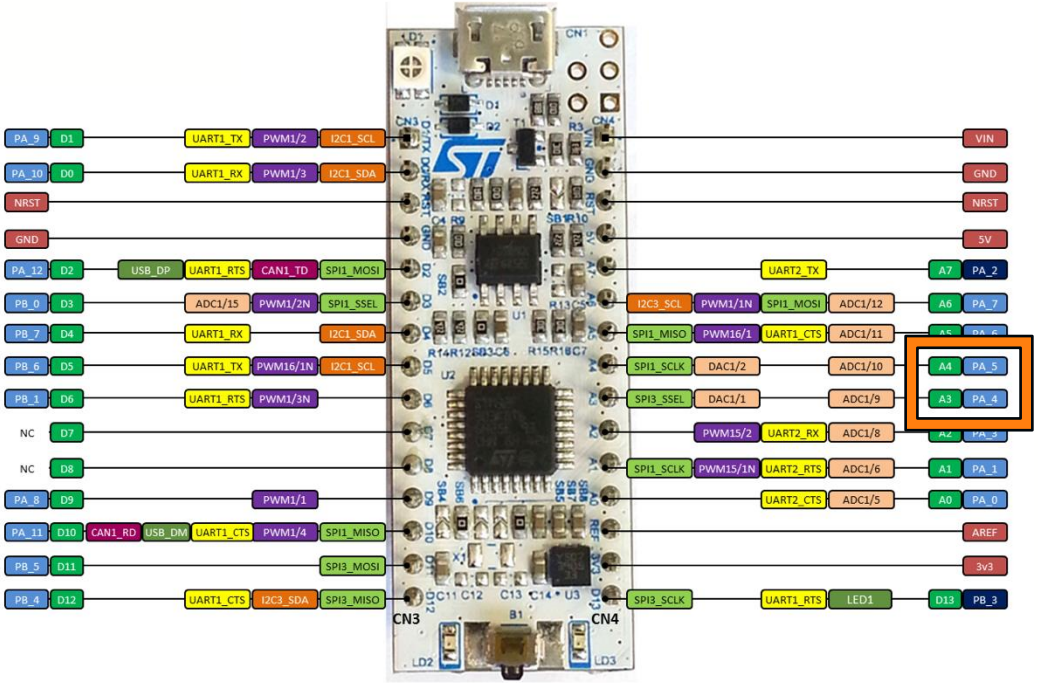
- Zero-order hold convers values to voltages
- The interpolator (I) “smooths” sharp transitions

STM32L4 Series DAC Summary

From Chapter 17 of the Reference Guide (RM0394)

- Two DAC modules that can be configured for 12 or 8-bit operation
- Sampling rate: 1.0 mega samples / second (Msps)
- Sample and hold mode for extremely low power operation (🌶️🌶️)
- Special modes for noise and triangular wave generation (🌶️🌶️)

NUCLEO L432KC DAC Pins



GPIO pins A3 and A4

- The dev board has 2 output pins that connect to a DAC

NUCLEO L432KC Board – DAC API

Function	Description
AnalogOut **name**(PinName pin)	Create an AnalogOut object, connected to the specified pin
void **write**(float value)	Set the output voltage, specified as a percentage (between 0.0 and 1.0) of the reference voltage
void **write_u16**(unsigned short value)	Set the output voltage, represented as an unsigned short in the range [0x0, 0xFFFF]
float **read**()	Return the current output voltage setting, measured as a percentage (between 0.0 and 1.0)
AnalogOut& **operator=**(float value)	Like write but use the = operator
operator **float()**	Like read but use the float typecast operator

Example: Using the DAC to Generate Specified Voltages

```
#include "mbed.h"

// Blinking rate in milliseconds
#define BLINKING_RATE     2000ms

int main() {
    DigitalOut led(LED1);
    AnalogOut aout1(PA_4);  // I am labeled as pin A3
    AnalogOut aout2(PA_5);  // I am labeled as pin A4

    unsigned int i;
    while (true) {
        led = !led;
        if (i % 2 == 0) {
            aout1 = 0.5;          // 3.33V / 2 = 1.67V
            aout2 = 0.5 / 3.33;   // 3.33V / 3.33 * 0.5 = 0.5V
        }
        else {
            aout1 = 1.0;          // = 3.33V
            aout2 = 1.0 / 3.33;   // 3.33V / 3.33 = 1.0V
        }
        i++;
        ThisThread::sleep_for(BLINKING_RATE);
    }
}
```

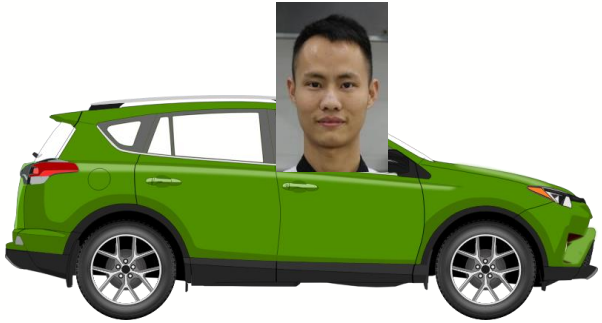
Of note:

- Pin A3 alternates between 1.67V and 3.33V
- Pin A4 alternates between 0.5V and 1.0V
- How small can we make BLINKING_RATE?

Part 3:

Direct Memory Access (DMA)

Food Delivery Model #1



Step 1:
Chef cooks food

Step 2:
Chef gets in car
and drives to you

Step 3:
You enjoy dinner!

Food Delivery Model #2

Step 1:
Chef cooks food
(begins next order)

Step 2:
Delivery person
transports food

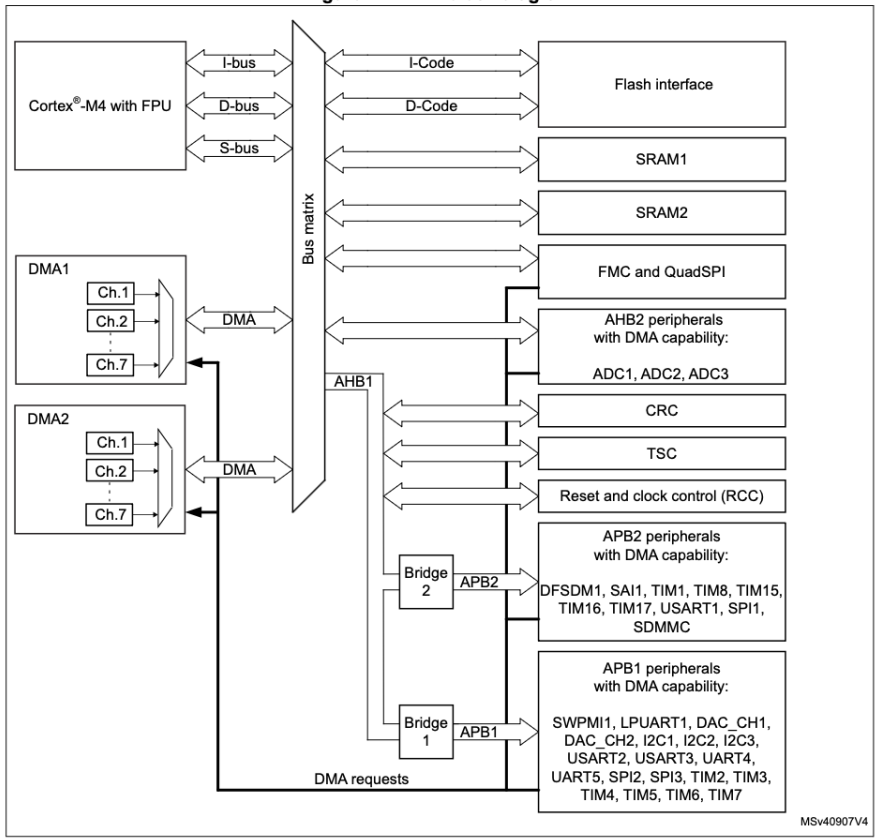
Step 3:
You enjoy dinner!

Bus Mastering

- Wasteful for the CPU to spend time moving 1's and 0's
- Peripherals can act as bus master (总线主控)
- Bus master acts as officer 🚦👮‍♂️ managing "bit traffic" 🚦 (总线主控器负责管理位流量)
- Direct Memory Access (DMA) controller is a peripheral dedicated to moving 1's and 0's on the system bus

STM32L4 DMA Controller

Figure 27. DMA block diagram



- The STM32L4 has 2 DMA controllers
- Each can control 7 data transfers between
  - Peripheral-to-memory
  - Memory-to-peripheral
  - Memory-to-memory
  - Peripheral-to-peripheral
- Mbed does not give direct access to DMA controllers

DAC Bit Delivery

- Goal: Output a waveform (任意波形) with the DAC

(See AN3126: How to use DAC for audio and waveform generation on STM32 MCUs and MPUs)

Figure 3. DAC interaction without DMA

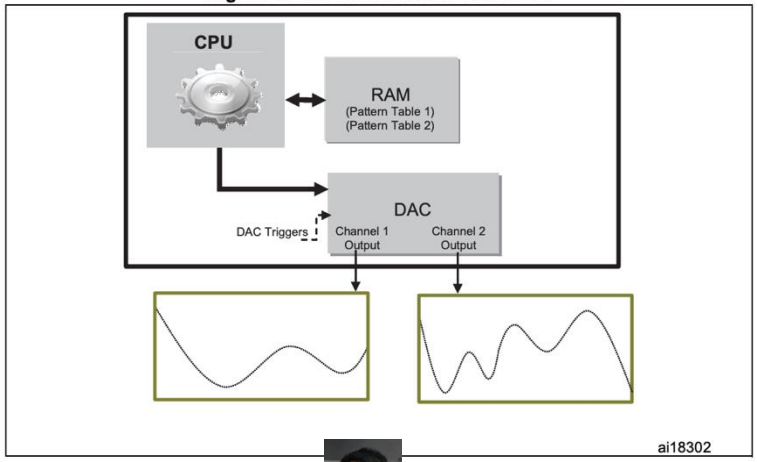
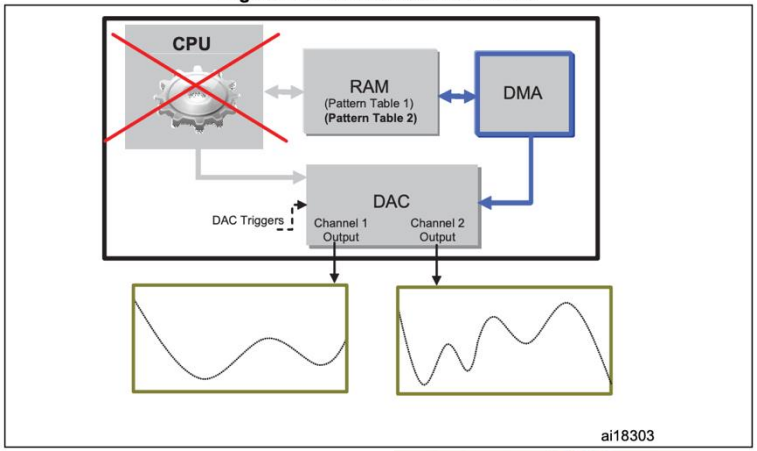
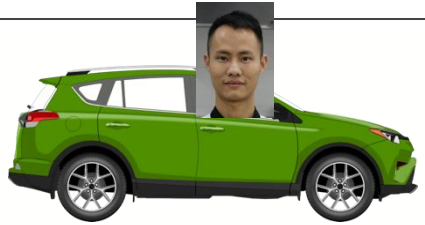


Figure 4. DAC interaction with DMA

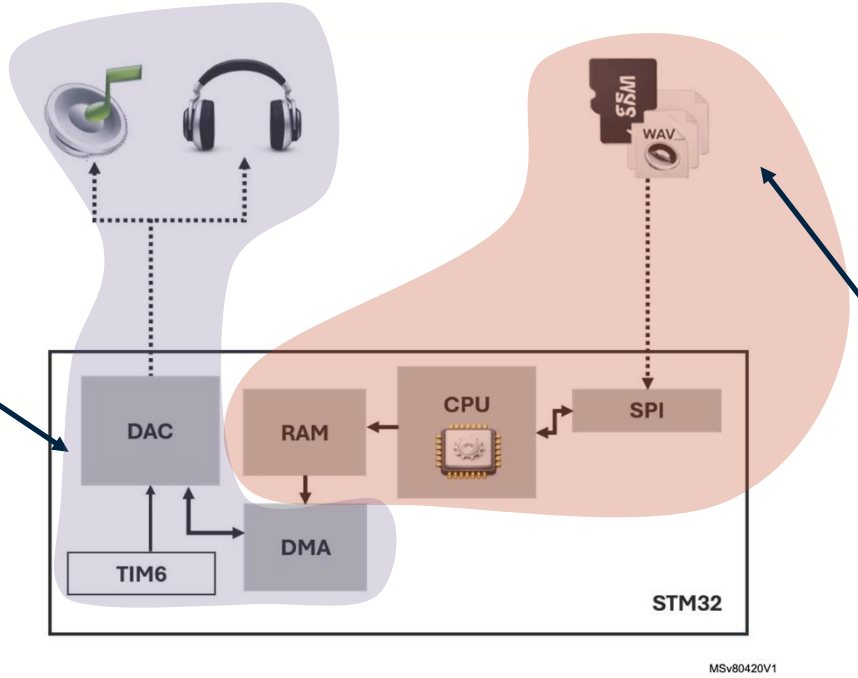




Audio Player Implementation

(See AN3126: How to use DAC for audio and waveform generation on STM32 MCUs and MPUs and UM1011 user manual)

DMA controller transfers RAM audio data to DAC for output



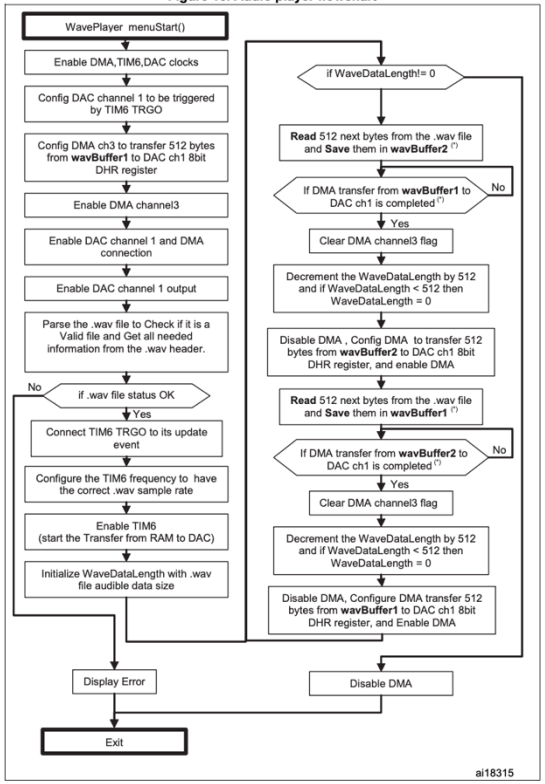
CPU reads .wav audio data (decoding samples if necessary) and transfers to RAM

DMA Comes with Complexity

```
// The following is pseudocode (伪代码)
AnalogOut aout(PA_4);
while (file.read(buffer)) {
    for (int i=0; i < CHUNK_SIZE; i++) {
        aout.write_u16(buffer[i]);
        wait_us(22); // Manual timing for ~44kHz
    }
}
```

(Timing gaps in this simple code will cause “click” and “pop” sounds “咔哒”声和“爆音”)

Figure 16. Audio player flowchart



When DMA transfers data from one SRAM buffer, the CPU transfers data from the microSD™ Flash memory to the other SRAM buffer.

DMA Frees the CPU

Figure 4. DAC interaction with DMA

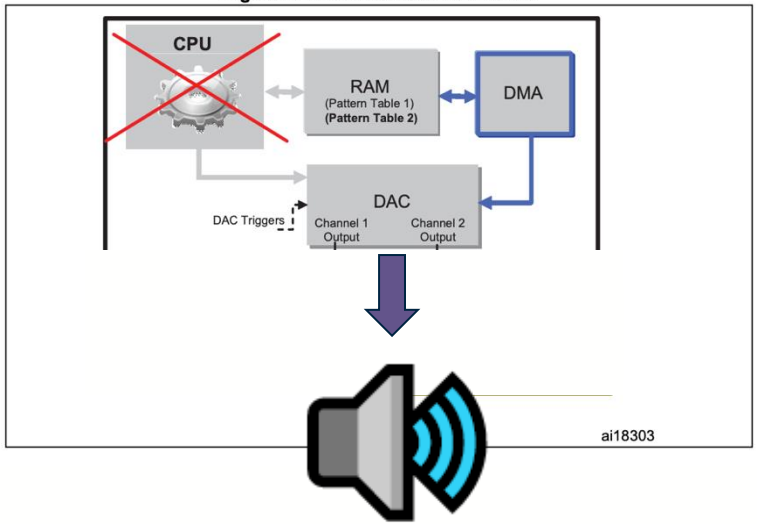
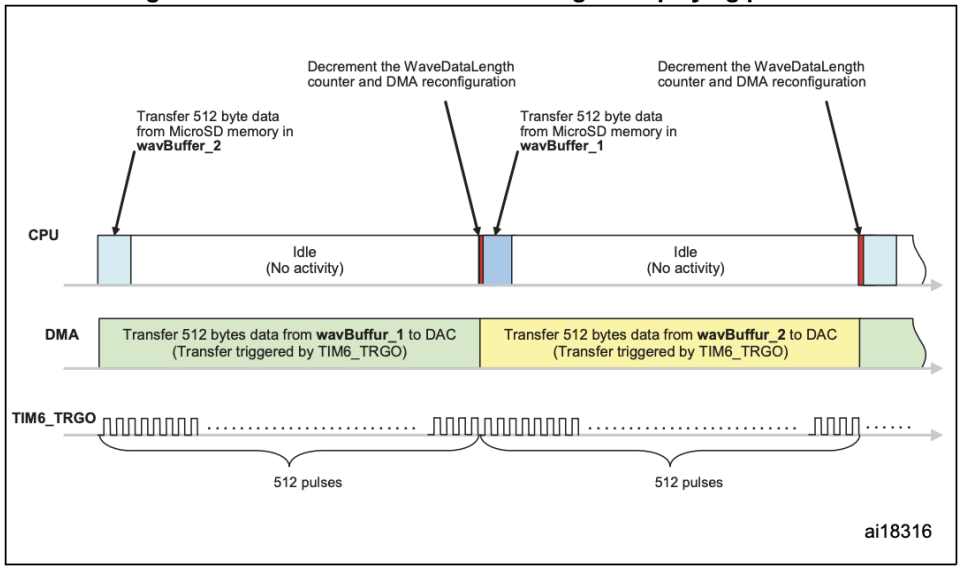


Figure 17. CPU and DMA activities during wave playing process



University of Glasgow

University of Glasgow 2013 Glasgow College, UESTC

WORLD CHANGING GLASGOW

Summary

- Interrupt request (IRQ) handler implementation in mbed
- DAC working principles
- DAC on the NUCLEO-L432KC
- Direct Memory Access fundamentals